Отчёт по лабораторной работе №7

Дисциплина: Архитектура компьютера

Пильщиков Никита Максимович

Содержание

1 Цель работы 5

2 Задание 6

3 Выполнение лабораторной работы 7

4 Выводы 17

Отчёт по лабораторной работе №7

3.1 Создание файла 7
3.2 Добавление текста программы 7
3.3 Создание исполняемого файла 8
3.4 Изменение текста программы 8
3.5 Запуск исполняемого файла 8
3.6 изменение текста программы 9
3.7 Создание файла 9
3.8 Создание файла 10
3.9 Создание файла 10
3.10 Проверка файла 11
3.11 Создание файла 11
3.12 Анализ первой строки 11
3.13 Анализ второй строки 12
3.14 Анализ третей строки 12
3.15 Удаление операнда 12
3.16 Проверка файла 12
3.17 Проверка изменений 12
3.18 Создание программы 13
3.19 Проверка файла 14
3.20 Создание новой программы для решения заданной функции . . . 15
3.21 проверка программы для первых переменных 15
3.22 Проверка программы для вторых переменных 16

Дисциплина: Архитектура компьютера

1 Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Задание

Разобраться в командах передачи управления или команды условного и безусловного перехода

3 Выполнение лабораторной работы

Так как у меня уже есть созданный катлог lab07 перехожу в него и создаю текстовый файл, ввожу в него текст команды из листинга (рис. 3.1).

```
[nmpiljthikov@nmpiljthikov lab07]$ touch lab7-1.asm
[nmpiljthikov@nmpiljthikov lab07]$ mcedit lab7-1.asm
```

Рис. 3.1: Создание файла

Текст из листинга (рис. 3.2).

```
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 3.2: Добавление текста программы

Создаю исполняемый файл и проверяю его работу (рис. 3.3).

```
[nmpiljthikov@nmpiljthikov lab07]$ ld -m elf_i386 -o lab7-1 lab7-1.o
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-1
Сообщение № 2
Сообщение № 3
```

Рис. 3.3: Создание исполняемого файла

Меняю текст программы так, чтобы она сначала выводила второе сообщение, затем первое (рис. 3.4).

```
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 3.4: Изменение текста программы

Создадим и запустим исполняемый файл для проверки правильности выполнения (рис. 3.5).

```
[nmpiljthikov@nmpiljthikov lab07]$ nasm -f elf lab7-1.asm
[nmpiljthikov@nmpiljthikov lab07]$ ld -m elf_i386 -o lab7-1 lab7-1.o
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-1
Сообщение № 2
Сообщение № 1
[nmpiljthikov@nmpiljthikov lab07]$ mcedit lab7-1.asm
```

Рис. 3.5: Запуск исполняемого файла

Изменим код так, чтобы программа выводила все три сообщения, но в обратном порядке (рис. 3.6).

```
%include 'in_out.asm' ; подключение внешнего фа
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:

jmp _label3

_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 1'
jmp _end

_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 2'
jmp _label1

_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintLF ; 'Сообщение № 3'
jmp _label2
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 3.6: изменение текста программы

Проверим правильность, создав и запустив исполняемый файл (рис. 3.7).

```
[nmpiljthikov@nmpiljthikov lab07]$ nasm -f elf lab7-1.asm
[nmpiljthikov@nmpiljthikov lab07]$ ld -m elf_i386 -o lab7-1 lab7-1.o
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
```

Рис. 3.7: Создание файла

Создаю файл lab7-2 и ввожу в него указанный текст программы (рис. 3.8)

```
[nmpiljthikov@nmpiljthikov lab07]$ touch lab7-2.asm
[nmpiljthikov@nmpiljthikov lab07]$ mcedit lab7-2.asm
```

Рис. 3.8: Создание файла

Ввод текста из листинга (рис. 3.9)

```
_start:
; ---------- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ---------- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ---------- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ---------- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ---------- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ---------- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в `max`
; ---------- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx,[B] ; иначе 'ecx = B'
mov [max],ecx
; ---------- Вывод результата
fin:
mov eax, msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax,[max]
call iprintLF ; Вывод 'max(A,B,C)'
call quit ; Выход
```

Рис. 3.9: Создание файла

Проверка исполнения на разных переменных B (рис. 3.10).

```
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-2
Введите B: 3
Наибольшее число: 50
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-2
Введите B: 70
Наибольшее число: 70
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-2
Введите B: 12
Наибольшее число: 50
```

Рис. 3.10: Проверка файла

#Изучение структуры файлов листинга

Создаю файл листинга для программы из файла lab7-2.asm (рис. 3.11).

```
[nmpiljthikov@nmpiljthikov lab07]$ nasm -f elf -l lab7-2.lst lab7-2.asm
```

Рис. 3.11: Создание файла

Ознакомлюсь с файлом и детально изучу три его строчки. Сперва рассмотрю строчку под номером 5, её адрес - 00000001, машинный код - 89C3, а mov ebx,eax - пересылает значение регистра ebx в регистр eax (рис. 3.12).

```
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-2
Введите B: 3
Наибольшее число: 50
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-2
Введите B: 70
Наибольшее число: 70
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-2
Введите B: 12
Наибольшее число: 50
```

Рис. 3.12: Анализ первой строки

Рассмотрим вторую строку под номером 33. В данной строчке 0000001D - её адрес, BB01000000 - её машинный код, а mov ebx,1 - пишет в регистр ebx значение

1 (рис. 3.13).

```
33 0000001D BB01000000          <1>     mov     ebx, 1
```

Рис. 3.13: Анализ второй строки

Рассмотри третью строчку. Здесь 00000008 - адрес строки, 40 - машинный код, inc ebx - увеличивает значение операнда ebx на 1 (рис. 3.14).

```
10 00000008 40                  <1>     inc     eax........
```

Рис. 3.14: Анализ третей строки

Открываю файл программы и убираю операнд max (рис. 3.15).

```
fin:
mov eax, msg2
call sprint ;.
mov eax,
call iprintLF ; Вывод max '(A,B,C)'
```

Рис. 3.15: Удаление операнда

Пытаюсь скомпилировать и запустить файл листинга, проверим какие файлы создались (рис. 3.16).

```
[nmpiljthikov@nmpiljthikov lab07]$ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:47: error: invalid combination of opcode and operands
[nmpiljthikov@nmpiljthikov lab07]$ ls
in_out.asm  lab7-1      lab7-1.o  lab7-2.asm  lab7-3.asm    report
Lab07       lab7-1.asm  lab7-2    lab7-2.lst  presentation
[nmpiljthikov@nmpiljthikov lab07]$ mcedit lab7-2.asm
```

Рис. 3.16: Проверка файла

Откроем файл lab7-2.lst и проверим, как выглядит наш код, что именно изменилось (рис. 3.17).

```
46 0000015E E8ACFEFFFF          call sprint ;.
47                              mov eax,
47          ******************   error: invalid combination of opcode and operands
48 00000163 E81EFFFFFF          call iprintLF ; Вывод max '(A,B,C)'
49 00000168 E86EFFFFFF          call quit ; Выход
```

Рис. 3.17: Проверка изменений

Под тем местом, где мы удалили операнд высветилась наша ошибка

#Задания для самостоятельной работы

Напишем программу, которая из 3 целачисленных значений выберет наименьшее “значения я буду подставялть из 6 варианта листинга в соотвествии с лабораторной №6” (рисю 3.18).

```
%include 'in_out.asm'
section .data

msga db 'Введите A: ',0h
msgb db 'Введите B: ',0h
msgc db 'Введите C: ',0h
msg1 db "Наименьшее число: ",0h
section .bss
min resb 10
A resb 10 ; резервируем место не
B resb 10 ; только под B,
C resb 10 ; но и под A и C
section .text
global _start
_start:
; ---------- Ввод A, B, C
mov eax,msga
call sprint
mov ecx,A
mov edx, 10
call sread
mov eax,msgb
call sprint
mov ecx,B
call sread
mov eax,msgc
call sprint
mov ecx,C
call sread
; ---------- Преобразование A,B,C из символов в числа
mov eax,A
call atoi ; Вызов подпрограммы перевода символа в число
mov [A],eax ; запись преобразованного числа в 'A'
mov eax,B
call atoi
mov [B],eax
```

Рис. 3.18: Создание программы

Проверим правильное функцианирование нашей программы (рис. 3.19).

```
%include 'in_out.asm'
section .data

msga db 'Введите A: ',0h
msgb db 'Введите B: ',0h
msgc db 'Введите C: ',0h
msg1 db "Наименьшее число: ",0h
section .bss
min resb 10
A resb 10 ; резервируем место не
B resb 10 ; только под B,
C resb 10 ; но и под A и C
section .text
global _start
_start:
; ---------- Ввод A, B, C
mov eax,msga
call sprint
mov ecx,A
mov edx, 10
call sread
mov eax,msgb
call sprint
mov ecx,B
call sread
mov eax,msgc
call sprint
mov ecx,C
call sread
; ---------- Преобразование A,B,C из символов в числа
mov eax,A
call atoi ; Вызов подпрограммы перевода символа в число
mov [A],eax ; запись преобразованного числа в 'A'
mov eax,B
call atoi
mov [B],eax
```

Рис. 3.19: Проверка файла

Убедившись в правильной работе своей программы, создаю новую. Создам программу, которая решает заданную функцию(Вариант 6) (рис. 3.20).

```
%include 'in_out.asm'
section .data
msgx db 'Введите x: ',0h
msga db 'Введите a: ',0h
msg1 db "Результат: ",0h

section .bss
x resb 10 ; резервируем место под x
a resb 10 ; под a
res resb 10 ; и под результат

section .text
global _start
_start:
; ------ Ввод значений x, a
mov eax,msgx
call sprint
mov ecx,x
mov edx,10
call sread
mov eax,msga
call sprint
mov ecx,a
call sread
; ------ Преобразование x, a в числа
mov eax,x
call atoi ; вызов подпрограммы перевода символа в число
mov [x],eax ; запись преобразованного числа в 'x'
mov eax,a
call atoi
mov [a],eax
; ------ Логическая часть программы
mov eax, [a]
mov ebx, [x]
cmp ebx, eax ; сравнение a и x
```

Рис. 3.20: Создание новой программы для решения заданной функции

Убедимся в верном написании программы, подставив заданные переменные (рис. 3.21).

```
[nmpiljthikov@nmpiljthikov lab07]$ nasm -f elf lab7-4.asm
[nmpiljthikov@nmpiljthikov lab07]$ ld -m elf_i386 -o lab7-4 lab7-4.o
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-4
Введите x: 2
Введите a: 2
Результат: 4
```

Рис. 3.21: проверка программы для первых переменных

Обязательно проверим работу программы для вторых переменных (рис. 3.22).

```
[nmpiljthikov@nmpiljthikov lab07]$ ./lab7-4
Введите x: 2
Введите a: 1
Результат: 10
```

Рис. 3.22: Проверка программы для вторых переменных

Отлично, программа работает без ошибок, ответ получен верный.

4 Выводы

В ходе выполнения лабораторной работы я изучил команды условного и безусловного переходов, а также приобрел навыки написания программ с использованием переходов, познакомилась с назначением и структурой файла листинга.